

Rapport d'analyse - Architecture Microservices



Welcome to the game!

<https://github.com/abdemeh/PokeDrafter>

Abdellatif EL-MAHDAOUI

Betsaleel CLOVIS

Alaa BOUGHAMMOURA

ICC ING3



1. Présentation du projet

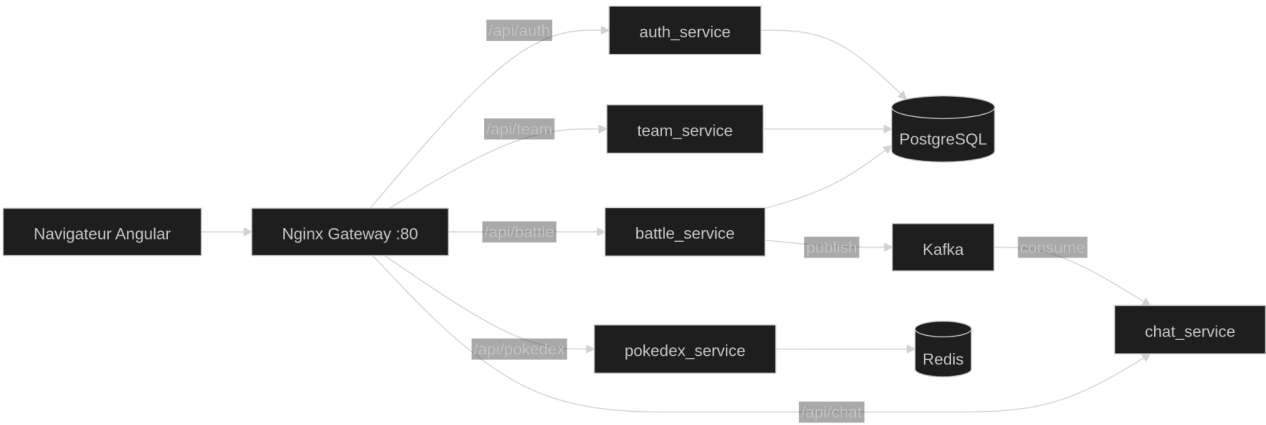
PokeDrafter est une application web multijoueur de combat Pokémon au tour par tour. Deux équipes s'affrontent : l'équipe Rouge contre l'équipe Bleue sur la base d'une formule de calcul de dommages par types F(A). On a voulu que chaque partie de l'appli soit indépendante et qu'on puisse en modifier une sans casser les autres, d'où le choix d'une architecture microservices.

On a aussi intégré un Pokédex avec cache Redis, un constructeur d'équipes, un chat en temps réel via WebSocket, et des notifications de combat via Kafka.

2. Architecture globale

Une architecture avec des microservices pour répondre aux besoins très différents de chaque domaine fonctionnel. L'authentification a besoin de transactions fiables (PostgreSQL), le Pokédex n'a besoin que d'un cache rapide (Redis), le chat n'a pas besoin de persistance du tout. Un monolithe aurait forcé un compromis global peu satisfaisant.

Le point d'entrée unique est un Nginx sur le port 80, qui dispatche vers les cinq services selon le préfixe d'URL. Les services backend ne sont jamais exposés directement à l'extérieur.



Service	Rôle	Stockage
auth_service	JWT, inscription, profil utilisateur	PostgreSQL
team_service	CRUD équipes de 6 Pokémon	PostgreSQL
battle_service	Moteur F(A), gestion des tours, Kafka producer	PostgreSQL
pokedex_service	Proxy PokéAPI + mise en cache	Redis
chat_service	WebSocket temps réel, Kafka consumer	En mémoire

3. Choix techniques

FastAPI

On a choisi FastAPI pour son support async natif : lors des appels vers PokéAPI ou la base de données, le serveur ne bloque pas. La validation automatique via Pydantic v2 évite d'écrire des vérifications manuelles à chaque endpoint, et la doc OpenAPI est générée sans effort. Flask serait trop synchrone pour ce qu'on faisait, Django trop lourd pour des microservices légers.

Kafka

Kafka sert à découpler battle_service et chat_service. Sans ça, un appel HTTP direct du service de combat vers le chat crée un couplage fort : si le chat est down, le tour échoue. Avec Kafka, battle_service publie un événement et passe à autre chose, le tour est enregistré même si la notification ne part pas. C'est un choix de résilience délibéré.

Redis

Les données Pokémon sont statiques dans le contexte du projet. Redis permet de mettre en cache une ressource PokéAPI (qui prend 200-500 ms à charger) et de la servir ensuite en moins de 5 ms. PostgreSQL aurait été surdimensionné pour ce cas d'usage purement lecture.

Nginx Gateway

Sans gateway, le frontend devrait connaître l'adresse de chaque service. Nginx centralise le routage par préfixe d'URL (/api/auth, /api/battle, etc.) et gère le proxying WebSocket, ce qui n'est pas trivial : il faut ajouter les headers Upgrade et Connection explicitement, sinon le handshake échoue (c'est d'ailleurs une des difficultés rencontrées).

JWT stateless

Chaque service vérifie la signature du token localement avec la SECRET_KEY partagée, sans appel réseau vers auth_service. Ça évite un point de défaillance unique : si auth_service est down, les requêtes déjà authentifiées continuent de fonctionner.

bcrypt

bcrypt est un algorithme adaptatif à coût variable avec sel aléatoire par hash. MD5 ou SHA-256 sont trop rapides pour du stockage de mots de passe, un attaquant peut tester des millions de combinaisons par seconde. Avec bcrypt, le coût est configurable et augmente avec la puissance de calcul disponible.

4. Services - points clés

battle_service - calcul de la puissance d'un pokemon

Le battle_service contient le moteur qui calcule la puissance d'un Pokémon A contre un Pokémon B. Le principe est simple : On regarde les types de A et les types de B, puis on applique les multiplicateurs officiels Pokémon (fort, faible, etc.). On combine ensuite ces effets pour obtenir un score final qui représente la puissance de A contre B. Pour éviter qu'un Pokémon avec un seul type soit désavantagé, on considère son type deux fois dans le calcul.

Après chaque tour de jeu, le résultat est sauvegardé en base de données. Un événement est aussi envoyé dans Kafka pour notifier les autres services (comme le chat). Si Kafka ne fonctionne pas, le tour est quand même enregistré : le jeu continue normalement, même sans notification.

chat_service - consumer Kafka résilient

La première version abandonnait le consumer dès le premier échec de connexion à Kafka. En pratique, au démarrage de la stack Docker Compose, Kafka met quelques secondes à être opérationnel, donc le consumer crashait systématiquement au boot.

La solution : une boucle while True avec backoff exponentiel (2s à 30s max) pour se reconnecter automatiquement. C'est un pattern classique pour les consumers Kafka dans un environnement où les services démarrent dans un ordre non garanti.

pokedex_service - cache Redis async

On utilise `httpx` (async) à la place de `requests` (synchrone) pour ne pas bloquer la boucle d'événements FastAPI lors des appels vers PokéAPI. Un appel synchrone dans un contexte async bloquerait tous les autres clients en attente.

5. Infrastructure

Docker Compose

Docker Compose garantit que tout le monde travaille dans le même environnement, quelle que soit la machine. Un seul docker compose up démarre les cinq services, PostgreSQL, Redis, Kafka et Nginx. On a ajouté des healthchecks pg_isready sur PostgreSQL avec depends_on: condition: service_healthy pour éviter que les services démarrent avant que la base soit prête, un problème classique qu'on a rencontré au début.

Kubernetes

Six manifests YAML dans infra/k8s/ permettent le déploiement sur cluster :

Fichier	Contenu
namespace.yaml	Namespace pokedrafter
postgres.yaml	Deployment + PVC 1 Gi
redis.yaml	Deployment + PVC 512 Mi
kafka.yaml	Zookeeper + Kafka Broker
services.yaml	Les 5 microservices applicatifs
gateway.yaml	Nginx exposé en NodePort :30080

Le réseau Docker est fermé : seul Nginx est accessible depuis l'extérieur. Les services backend communiquent entre eux via le réseau interne Docker, sans exposition directe.

6. Sécurité

Les choix de sécurité sont cohérents avec l'architecture stateless :

- JWT stateless : vérification locale de la signature, pas de base de sessions centralisée
- bcrypt : sel aléatoire, coût adaptatif, résistant aux attaques par rainbow tables
- Pydantic : validation automatique à l'entrée des endpoints, rejet HTTP 422 avant même d'atteindre la logique applicative
- Protection IDOR : le user_id est toujours extrait du JWT côté serveur, jamais lu depuis le corps de la requête
- Réseau Docker fermé : seul Nginx est exposé sur l'hôte

On a aussi identifié les limites actuelles, qu'on n'a pas eu le temps de traiter dans ce projet :

- Pas de blacklist de tokens JWT après logout - un token reste valide jusqu'à expiration
- Le WebSocket n'est pas authentifié par token à la connexion
- CORS configuré de façon permissive en développement

7. Répartition du travail

Prénom	Périmètre de travail
Alaa	auth_service (JWT, bcrypt, inscription), team_service (CRUD équipes JSON), pokedex_service (cache Redis, httpx async)
Betsaleel	battle_service (moteur F(A), Kafka producer, WebSocket PvP, /join, /forfeit), chat_service (WS, consumer Kafka avec retry), Docker Compose, Nginx, manifests Kubernetes
Abdelatif	Frontend Angular 17 - composants standalone, routing, JWT interceptor, client WebSocket

8. Conclusion

Chaque choix technique du projet répond à une contrainte précise. FastAPI async pour éviter le blocage sur les I/O. Kafka pour découpler battle_service et chat_service sans les rendre dépendants l'un de l'autre. Redis pour un cache statique rapide, sans sortir l'artillerie lourde PostgreSQL. JWT stateless pour ne pas créer un point de défaillance unique sur toutes les requêtes authentifiées. Nginx pour un point d'entrée unique qui protège les services backend.

Les difficultés rencontrées : configuration Kafka silencieuse, lazy loading async, race condition au démarrage Docker, illustrent bien les contraintes concrètes d'une architecture distribuée. Ce ne sont pas des problèmes théoriques : on les a tous rencontrés et résolus en cours de développement.

Le projet est déployable en une commande (docker compose up) ou sur cluster Kubernetes via les manifests fournis. Le flux Kafka est validé end-to-end, les cas de combat couvrent les principaux types, et les transitions d'état des salles fonctionnent correctement.